

# Three entry points, not three APIs

Evaluation of the abstraction-levels review: the optimization insight adopted and elevated, bounded contexts adopted with a reframing, and one recommendation rejected because it would reopen the hole the whole model was built to close.

---

## Strongest contribution

Payment is the unit of optimization

## Principal rejection

Attempt as a client write API

## Reframing

Bounded contexts, not levels

## Net

Right instinct, wrong third object

## SECTION 1

# Verdict

The review's instinct is right on every point. One of its conclusions is wrong, and it happens to be the load-bearing one.

## Adopted

| RECOMMENDATION                                                                    | VERDICT | REASONING                                                                                                                  |
|-----------------------------------------------------------------------------------|---------|----------------------------------------------------------------------------------------------------------------------------|
| Payment is the object the Decision Platform optimizes                             | ADOPT   | The best new idea in the review and the sharpest argument yet for why the separation matters. Elevated to its own section. |
| Order is the commerce abstraction above payments, not another payment abstraction | ADOPT   | Already canonical. The review states it more crisply than we have, and the wording should be borrowed.                     |
| Order is optional; a payment may exist without one                                | ADOPT   | Matches the branch model. A client with their own order system never creates one here.                                     |
| Composable, not prescriptive — don't force Order to parent every payment          | ADOPT   | Correct, and it is the same conclusion the integration-model correction reached from the opposite direction.               |
| Intent is the business objective; Attempt is the execution                        | ADOPT   | Verbatim the canonical distinction. No change.                                                                             |
| Order enables partial fulfilment where payment objects cannot                     | ADOPT   | Fulfilment and collection lifecycles genuinely differ. Good worked example.                                                |
| Order gives refund, dispute and allocation a natural home                         | ADOPT   | Consistent with the v3 refund and dispute model.                                                                           |

## Amended

| RECOMMENDATION                            | VERDICT | REASONING                                                                                                                           |
|-------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------|
| Three levels of abstraction               | AMEND   | Right idea, wrong metaphor. They are not levels of sophistication but <b>bounded contexts</b> , independently enterable. Section 3. |
| The 18-row capability table               | AMEND   | Useful artefact with four errors, including one that conflates Payment and Intent as separate objects. Corrected in Section 6.      |
| "PaymentIntent" as the client-facing name | AMEND   | The canonical model locked this: internal concept is the payment intent, the external API says <code>Payment</code> . Section 7.    |

## Rejected

| RECOMMENDATION                                                                  | VERDICT | REASONING                                                                                                                                                                       |
|---------------------------------------------------------------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>POST /payment_attempts</code> as a client-facing API for simple merchants | REJECT  | Reopens the idempotency hole the Payment object exists to close, contradicts design rule 3, and buys no simplicity that a one-call Payment does not already provide. Section 4. |

#### THE ONE-LINE SUMMARY

The review correctly identifies that clients should be able to enter the model at more than one place. It then picks the wrong third place. The three entry points are **Order**, **one-call Payment** and **explicit Payment** — not Order, Intent and Attempt.

## SECTION 2

# The strongest contribution

One sentence in the review is worth more than the rest of it, and it has not appeared in any of our documents.

#### ADOPTED AND ELEVATED

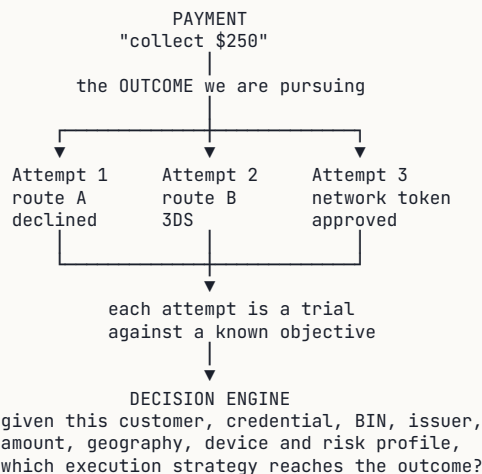
**The Payment is the thing you optimize. The Attempt is the experiment.**

A Payment has a desired outcome. Every attempt beneath it is a datapoint about how to reach that outcome. This is the cleanest statement yet of why the Decision Engine needs the payment object — and it is a better argument than anything currently written down.

## 2.1 Why this matters more than it first appears

Every prior argument for the durable payment object has been **defensive**: it prevents double charges, it survives interruptions, it gives disputes a parent. Those are all true and all about avoiding harm.

This one is **offensive**. It says the payment object is what makes the platform able to learn.



## 2.2 Why an attempt alone cannot be optimized

An attempt on its own has no objective attached to it. It succeeded or it did not. Without the payment above it, there is no way to ask the question that matters:

| QUESTION                                     | NEEDS                                                                             |
|----------------------------------------------|-----------------------------------------------------------------------------------|
| Did this attempt succeed?                    | The attempt alone                                                                 |
| <b>Did we eventually collect the money?</b>  | The payment                                                                       |
| <b>How many tries did it cost us?</b>        | The payment                                                                       |
| <b>Would route B first have been better?</b> | The payment — a counterfactual needs a fixed objective to be counterfactual about |
| <b>Is our routing policy improving?</b>      | The payment, aggregated                                                           |

#### THE CONSEQUENCE FOR THE DECISION PLATFORM

Counterfactual estimation, simulation and experimentation all require a **stable objective** to measure alternative strategies against. The payment is that objective. A platform that exposed only attempts could report which attempts succeeded, but could never answer whether a different strategy would have collected the money — because there would be nothing that represents "the money we were trying to collect."

## SECTION 3

# Bounded contexts, not levels

The review's framing is right in substance and misleading in metaphor.

## 3.1 The problem with "levels"

"Three levels of abstraction" implies a ladder, with simple clients at the bottom and sophisticated clients at the top. That framing has already caused one defect in this programme: it produced a per-use-case path assignment that would have forced a single retailer onto two different integrations.

#### WHAT A LADDER IMPLIES, WRONGLY

That a client who starts simple is on a lower rung and should climb. In practice a restaurant group with a sophisticated point-of-sale estate may use the simplest call in the API, and an unsophisticated merchant with no systems at all may need the richest object we offer. **Sophistication of the client does not predict which object they use.** What they already own does.

## 3.2 The better frame

These are three **bounded contexts**, each with its own domain, its own language and its own reason to exist. A client enters whichever contexts their business actually occupies.

COMMERCE CONTEXT  
what is being bought  
owned by: merchant or us

COLLECTION CONTEXT  
what we are trying to collect  
owned by: always us

EXECUTION CONTEXT  
what we actually did  
owned by: always us

ORDER  
line items · tax · fulfilment · returns  
optional — only if we hold the record

PAYMENT  
durable objective · survives attempts  
always exists, sometimes unnamed

ATTEMPT  
route · credential · response · latency  
never client-writable — see Section 4

### 3.3 What follows from the reframing

- **A client can occupy the collection context without the commerce context.** They have their own order system; they enter at Payment. This is the OMS branch, and the review reaches it independently.
- **A client can occupy the commerce context and never name the collection context.** They create an Order; the Payment exists but they never touch it.
- **No client occupies the execution context as a writer.** That is the one the review gets wrong, and it is the subject of the next section.
- **Moving between contexts is not promotion.** A merchant who starts at Order and later reaches into payments has not graduated. They have simply begun occupying a second context.

#### THE WORDING WORTH BORROWING VERBATIM

"Order should not be another payment abstraction. It should be the commerce abstraction above payments." That is the clearest single sentence anyone has written about the Order object in this programme, and it should replace the longer explanation currently in the canonical model.

## SECTION 4

# Why Attempt must not be a client write API

The review proposes `POST /payment_attempts` for simple merchants. This is the one recommendation to refuse, and the reasoning matters more than the refusal.

## 4.1 The proposal, fairly stated

The review positions an attempt API as: *"I know exactly what payment I want to execute. Just execute it."* It claims seven benefits — a very simple mental model, maximum control, good fit for simple checkout, a natural retry model, clean transaction analytics, compatibility with orchestration, and lower integration complexity.

The instinct behind it is correct: **some clients genuinely want one call and no concepts.** That need is real and must be served. The proposed mechanism is what fails.

## 4.2 Five reasons it fails

### REASON 1

#### It reopens the idempotency hole

The entire case for the durable payment object rests on one fact: without it, no server-side object exists before the first network message, so a retry has nothing to key against. If the client creates the attempt, the client is back to inventing an idempotency key and hoping it is stable across their own infrastructure retries, load-balancer timeouts and queue redeliveries. **This is the exact defect the Payment object was introduced to eliminate.**

#### REASON 2

### "Natural retry model" describes the failure, not a feature

The review lists as a benefit: "a new attempt can be created after a decline." But a new attempt created **by the client** is indistinguishable from a duplicate submission. The platform cannot tell a deliberate retry from a double-click, because nothing above the attempt says the two belong to one intended collection. Retry safety is not a client discipline we should be relying on.

#### REASON 3

### The simplicity saving is illusory

Compare the two calls honestly. They are the same shape, the same length, and the same number of round trips — but only one of them creates something durable.

|                                   |                                                          |
|-----------------------------------|----------------------------------------------------------|
| PROPOSED                          | ALREADY AVAILABLE                                        |
| POST /payment_attempts            | POST /payments                                           |
| { "amount": 12500,                | { "amount": 12500,                                       |
| "currency": "USD",                | "currency": "USD",                                       |
| "payment_method": "pm_123",       | "payment_method": "pm_123",                              |
| "merchant_reference": "INV-123" } | "order_ref": "INV-123",                                  |
|                                   | "confirm": true }                                        |
| → an orphan execution             | → a durable Payment, with the attempt created beneath it |

There is no integration complexity to save. The developer writes the same thing either way and gets a durable anchor for free in one of the two cases.

#### REASON 4

### It contradicts a locked design rule

Canonical design rule 3: "Payment is not Attempt. Payment is durable; attempts are executions. Three attempts must never become three charges." If clients create attempts directly, three attempts **can** become three charges. The rule and the proposal cannot both stand.

#### REASON 5

### It breaks the evidence graph

The dispute evidence graph assembles payment proof, authentication, device, session and fulfilment because all of them hang off a common parent. A client-created orphan attempt has no parent. When a dispute arrives ninety days later against an attempt that was never part of a payment, there is nothing to assemble.

## 4.3 The review's own table concedes the point

Three rows of its capability comparison score the attempt column as the weak one, and each is precisely a responsibility a PSP is supposed to absorb rather than return to the client.

| CAPABILITY               | ATTEMPT COLUMN SAYS | WHICH MEANS                                                                        |
|--------------------------|---------------------|------------------------------------------------------------------------------------|
| Stable payment lifecycle | Limited             | The client reconstructs state from individual transactions                         |
| Retry management         | Manual / client     | The client writes retry logic without knowledge of scheme ceilings or advice codes |

| CAPABILITY        | ATTEMPT COLUMN SAYS   | WHICH MEANS                          |
|-------------------|-----------------------|--------------------------------------|
| Multiple attempts | <i>Client-managed</i> | The client owns duplicate prevention |

A document arguing for an attempt-level API has written down, in its own comparison, that the attempt-level API hands three hard problems back to the merchant.

## 4.4 What we do instead

THE RULE

**Attempt is a read surface, never a write surface.**

Clients may read every attempt — route taken, credential used, issuer response, latency, decision rationale. They may never create one. The simple-merchant need the review identifies is served by a one-call Payment, which is the same ergonomics with a durable anchor.

|                                                            |                                                 |
|------------------------------------------------------------|-------------------------------------------------|
| CLIENT MAY                                                 | CLIENT MAY NOT                                  |
| POST /payments (confirm: true)<br>one call, durable object | POST /payment_attempts<br>orphan execution      |
| GET /payments/{id}/attempts<br>full decision trace         | create, modify or replay<br>an attempt directly |

## SECTION 5

# The corrected entry-point model

Three places a client can enter. None of them is the execution context.

|                                                             |                                    |
|-------------------------------------------------------------|------------------------------------|
| ENTRY A – COMMERCE                                          | for clients with no order system   |
| POST /orders                                                |                                    |
| POST /orders/{id}/checkout                                  |                                    |
| → we hold the order; Payment and Session created beneath it |                                    |
| ENTRY B – COLLECTION, one call                              | for clients who want a single call |
| POST /payments { amount, currency, payment_method,          |                                    |
| order_ref, confirm: true }                                  |                                    |
| → durable Payment created and confirmed in one round trip   |                                    |
| → this is the case the review tried to serve with attempts  |                                    |
| ENTRY C – COLLECTION, explicit                              | for delayed capture, multi-tender, |
| POST /payments                                              | cross-channel continuation,        |
| POST /payments/{id}/confirm                                 | authentication, marketplaces       |
| POST /payments/{id}/capture                                 |                                    |
| <hr/>                                                       |                                    |
| READ-ONLY, available from every entry point                 |                                    |
| GET /payments/{id}/attempts                                 | route · credential · response      |
|                                                             | latency · decision rationale       |

## 5.1 Mapping the review's client types onto the corrected model

| CLIENT THE REVIEW NAMES                     | REVIEW PROPOSES        | CORRECTED                                                               |
|---------------------------------------------|------------------------|-------------------------------------------------------------------------|
| Simple merchant, direct authorization       | POST /payment_attempts | <b>Entry B.</b> Same single call, durable object, no orphan executions. |
| Server-to-server payments                   | POST /payment_attempts | <b>Entry B.</b> Identical ergonomics.                                   |
| E-commerce with 3DS, retries, async methods | POST /payment_intents  | <b>Entry C,</b> named <code>Payment</code> rather than intent.          |
| Marketplace, travel, retail, split tender   | POST /orders + intents | <b>Entry A or C,</b> depending on who holds the order record.           |
| Client wanting execution visibility         | Attempt API            | <b>Read surface</b> on any entry point.                                 |

### WHAT THE CORRECTION PRESERVES

Every client type the review identifies is still served, and served with the same number of API calls. Nothing is taken away. The only change is that the simplest path creates a durable object instead of an orphan one — at no cost to the developer.

## SECTION 6

# The capability matrix, corrected

The review's comparison table is a genuinely useful artefact. Four rows are wrong and the column headings mislead.

## 6.1 What was wrong

| ROW                                                                       | PROBLEM                                                                                                                                                                            |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Refund aggregation</b><br>"Payment-level / Intent-level / Order-level" | Treats Payment and Intent as two different objects. Under the canonical model they are one object with an internal and an external name. The row has three columns for two things. |
| <b>Subscription</b><br>"Possible / Excellent / Useful"                    | Overstates Order. In the canonical model a subscription runs Invoice → Payment; Order is not involved at all. "Useful" should be "not used".                                       |
| <b>Natural retry model</b> (attempt column)                               | Scored as a benefit. It is the duplicate-charge exposure described in Section 4.                                                                                                   |
| <b>Lower integration complexity</b> (attempt column)                      | Scored as a benefit. A one-call Payment is the same complexity. There is no saving to score.                                                                                       |

## 6.2 The corrected matrix

Columns reframed: Attempt is what you **see**, not what you build on.

| CAPABILITY                | ORDER       | PAYMENT          | ATTEMPT (READ)    |
|---------------------------|-------------|------------------|-------------------|
| Execute a collection      | Indirect    | <b>Direct</b>    | Records it        |
| Single-call checkout      | Two calls   | <b>One call</b>  | n/a               |
| Durable payment lifecycle | Via payment | <b>Native</b>    | n/a               |
| Authentication lifecycle  | Via payment | <b>Native</b>    | Shows the outcome |
| Retry and replan          | Automatic   | <b>Automatic</b> | Shows each try    |

| CAPABILITY                 | ORDER          | PAYMENT            | ATTEMPT (READ)      |
|----------------------------|----------------|--------------------|---------------------|
| Routing decisions          | Automatic      | Automatic          | Shows the rationale |
| Idempotency anchor         | Present        | Present            | n/a                 |
| Multiple payments per sale | Native         | Client coordinates | n/a                 |
| Partial payments           | Native         | Client coordinates | n/a                 |
| Partial fulfilment         | Native         | Not modelled       | n/a                 |
| Line items                 | Native         | Reference only     | n/a                 |
| Refund resolution          | Across tenders | Per payment        | n/a                 |
| Marketplace and split      | Native         | Via obligations    | n/a                 |
| Subscription               | Not used       | Via invoice        | Shows dunning tries |
| Invoice collection         | Optional       | Native             | Shows each try      |
| Integration simplicity     | Medium         | High               | n/a                 |
| Commerce abstraction       | High           | Low                | None                |
| Execution visibility       | On request     | On request         | Complete            |

Highlighted cell in each row marks where that capability is best served. Order leads on seven, Payment on nine, Attempt on two — and the two Attempt leads on are both about seeing, not doing.

## SECTION 7

# Terminology and competitive claims

Two smaller corrections, one of which has already been decided.

## 7.1 Payment, not PaymentIntent

The review uses `PaymentIntent` as the client-facing name throughout. The canonical model settled this: the payment intent remains a valid **internal** concept, and the external API calls it `Payment`.

| CONCEPT                   | REVIEW SAYS                 | CANONICAL                                                  |
|---------------------------|-----------------------------|------------------------------------------------------------|
| Durable collection object | <code>PaymentIntent</code>  | <code>Payment</code> externally, payment intent internally |
| Execution record          | <code>PaymentAttempt</code> | <code>Attempt</code> — agreed                              |
| Commerce record           | <code>Order</code>          | <code>Order</code> — agreed                                |

This is not pedantry. The review's own argument supports the canonical choice: it notes that an enterprise developer thinks *"I have an order and I need to collect payment,"* not *"I need to instantiate a payment intent."* Having made that observation, the document then uses the intent vocabulary anyway.

## 7.2 The competitive claims

The review makes two claims about Adyen. Both are consistent with what was verified earlier in this programme against published documentation, and neither has been independently re-checked for this note.

| CLAIM                                                                                                                | STATUS                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Adyen distinguishes the initial payment request from subsequent payment-detail steps when further action is required | Consistent with the imperative / payments model verified earlier. Not re-checked.                                                           |
| Adyen's integration documentation has merchants update their own order-management system after the payment flow      | Consistent, and notable — it is independent support for the branch model. Adyen assumes the merchant owns the order record. Not re-checked. |

#### WORTH NOTICING

The second claim is the more interesting one. If Adyen's own documentation instructs merchants to update **their** order system after payment, that is an incumbent confirming the premise of our branch model: a client with an order system should not be asked to keep a second one. Worth verifying properly before it is used in any external material.

## SECTION 8

# What this changes

Four edits to existing documents. Nothing in the canonical model is reversed.

| DOCUMENT                      | CHANGE            | DETAIL                                                                                                                                                                  |
|-------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Canonical Architecture Model  | Add               | New subsection under the Decision Engine: <i>the Payment is the unit of optimization, the Attempt is the experiment</i> . Section 2 of this note supplies the text.     |
| Canonical Architecture Model  | Replace wording   | Swap the Order explanation for the review's sentence: " <i>Order should not be another payment abstraction. It should be the commerce abstraction above payments.</i> " |
| Canonical Architecture Model  | Add a design rule | Rule 9: <b>Attempt is a read surface, never a write surface.</b> Currently implied by rule 3; the review shows it needs to be explicit.                                 |
| When Is A Payment Needed      | Reframe           | Replace "levels" language with bounded contexts, and add Entry B — the one-call Payment — as an explicit named path rather than a footnote.                             |
| Order or Payment client guide | No change         | Already branch-based rather than ladder-based. The review's framing agrees with it.                                                                                     |

## 8.1 The new design rule, stated for the canonical model

### DESIGN RULE 9

#### **Attempt is a read surface, never a write surface**

Clients may read every attempt, including route, credential, issuer response, latency and decision rationale. Clients may never create, modify or replay one. An attempt exists only as a child of a Payment, because an execution without a stated objective cannot be made idempotent, cannot be optimized, and cannot carry dispute evidence.

## 8.2 What is not changing

- **The four concepts stand.** Commerce, Payment, Interaction, Money, with Obligation cutting across. No fifth concept is introduced.
- **The state models stand.** No change to Order, Payment, Session or Obligation lifecycles.
- **The branch model stands.** Whether a client holds their own order record remains the first question, and the review independently reaches the same conclusion.
- **The eight design rules stand.** Rule 9 is additive and makes rule 3 explicit rather than amending it.

### BOTTOM LINE

The review is right that clients should enter the model at more than one place, right that Order is a commerce abstraction rather than a payment one, and right — more usefully than it seems to realise — that the Payment is what the Decision Platform optimizes. It is wrong that the third entry point should be the attempt. **Adopt the optimization insight, adopt the composability, reframe levels as bounded contexts, and make explicit the rule the proposal would have broken.**